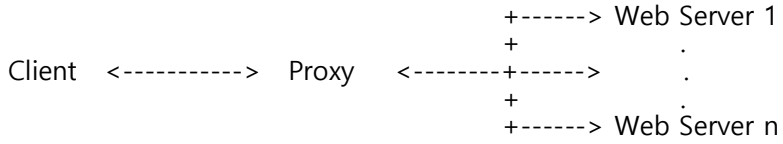
Computer Network: Proxied Web Server

Fall 2026

Client와 Web server 사이에 서버의 대리인 역할을 하는 proxy를 배치하여 사용자 요청과 그에 대한 결과가 중계되도록 하는 형태의 웹 서버 구조를 proxied Web server라고 한다. 이러한 proxy 구조를 갖는 이유로는 load balancing, caching, 보안 및 익명성 등의 이점을 누릴 수 있기 때문이다.



이번 학기의 프로그래밍 과제는 concurrent Web server와 reverse proxy 구현의 두 단계로 수행되는데, HTTP message 처리, concurrent server 구조, reverse proxy와 간단한 L7 load balancing 기능을 주요 내용으로 포함한다. 과제의 구현은 Java 프로그래밍 환경에서 이루어져야 하며, Java SE의 기본 패키지만 사용 가능하며 http package나 타 extension package의 사용은 허용되지 않는다.

클라이언트 역할 프로그램으로는 Web Browser, curl command, Postman (https://www.postman.com)을 이용할 수 있다.

단계 1: Multi-threaded Web Server 구현

1.1. HTTP 프로토콜 기본 기능 구현

Request와 response message의 교환으로 이루어지는 HTTP 프로토콜을 구현한다. 지원해야 하는 method는 GET, PUT, POST이며, 이후에 서술되는 기능을 지원하기 위해 필요한 header type, status code, body type 등을 지원해야 한다.

1. TCP 서버 소켓을 열고 특정 포트(예: 8080)를 바인딩하여 클라이언트 연결을 기다린다.
2. 클라이언트가 보낸 HTTP request line과 header line을 읽어 method, URI, HTTP version, Content-Length 등 메시지 내용을 파싱한다.
3. 웹 document root는 ./docs/public/로 한다. 요청 URI에는 해당하는 정적 파일을 이 directory에서 읽어 반환한다. 예를 들어 GET /index.html request에 대해서 ./docs/public/index.html을 반환한다.

4. HTTP response는 status line과 필요한 header를 작성한 뒤 body를 byte stream으로 전송한다. 정상적인 file response에는 적절한 Content-Type, Content-Length, Connection: close header를 포함한다. HTTP request-response message exchange가 끝나면 TCP connection을 종료하며, persistent connection과 pipelining은 필수 구현 사항이 아니다.
5. 요청한 resource가 존재하지 않거나 request를 정상적으로 처리할 수 없는 경우에 404 Not Found, 400 Bad Request, 405 Method Not Allowed 등 상황에 맞는 HTTP error response를 반환한다.

개발 웹 서버 프로그램을 "java MyWebServer --port 8080" command로 수행하면 local machine의 8080 port를 사용하게 된다.

1.2. 멀티스레드 기반의 Concurrent Web Server

위의 기본 웹 서버를 확장하여 GET/PUT/POST 메서드를 개발하고, 복수 클라이언트를 동시에 지원하는 concurrency 기능을 구현한다.

1. Multi-threading 또는 Thread Pool을 도입하여 복수의 브라우저나 클라이언트가 동시에 접속할 수 있도록 웹 서버를 확장한다. 즉, 이를 위해 직접 threaded 서버를 구현하거나 java.util.concurrent.ExecutorService을 이용해 스레드 풀을 이용할 수도 있다.
2. GET /filename=... 구현한다. 즉, ./docs/public/ 디렉토리에서 filename에 해당하는 파일을 읽어 client에게 반환한다.
3. HTTP PUT 메소드를 구현한다. 업로드 파일은 ./docs/upload/ 디렉토리에 저장한다. 저장할 filename은 URI에 직접 지정되며, request body 전체에는 업로드할 file의 binary data가 포함된다. 복수의 worker thread가 동일한 target file을 동시에 write하지 않도록 file write operation에 대해 synchronized access를 적용한다. 다음은 PUT request message의 예시이다.

```
PUT /upload/my_cat.jpg HTTP/1.1
Host: localhost:8080
Content-Type: application/octet-stream
Content-Length: <file size in bytes>

<binary file data>
```

- 저장 위치는 ./docs/upload/my_cat.jpg 이다.
- Content-Length 에 지정된 byte 수만큼 request body 를 읽어 파일로 저장한다.
- 새 파일 생성은 201 Created, 기존 파일 대체는 200 OK 로 응답한다.

4. HTTP POST 메소드와 multipart/form-data 형식의 file upload를 구현한다. 업로드 파일은 ./docs/upload/ 디렉토리에 저장한다. 서버는 Content-Type header에서 boundary 값을 얻고, request body에서 해당 boundary를 기준으로 multipart part를 구분한다. Content-Disposition의 name="file"인 file part에서 filename을 얻어 실제 file data를 byte 단위로 저장한다. 복수의 worker가 동일한 target file을 동시에 write하지 않도록 PUT과 유사한 synchronization 기법을 적용한다. 본 과제의 필수 구현 범위는 한 request에서 name="file"인 file part 하나를 처리하는 것으로 한다. 여러 file part를 포함하는 multi-file upload는 필수 구현 범위가 아니다.

   다음은 Postman에서 Body → form-data를 선택하고, key를 file, type을 File로 설정한 뒤 sample.png를 선택했을 때 전송되는 request의 구조를 단순화하여 나타낸 예시이다. 실제 boundary 문자열과 Postman이 자동으로 추가하는 일부 header는 실행 환경에 따라 달라질 수 있다.

```
POST /upload HTTP/1.1

Host: localhost:8080
Content-Type: multipart/form-data; boundary=Boundary123
Content-Length: <bytes>

--Boundary123 ← part 시작
Content-Disposition: form-data; name="file"; filename="sample.png"
Content-Type: image/png

<binary file data>
--Boundary123--  ← multipart 종료
```

- Boundary 를 이용하여 multipart 의 part header 와 file body 를 구분한다.
- Content-Disposition 에서 filename 을 추출하고 실제 파일 데이터는 바이트 단위로 저장한다.
- file part 가 없거나 multipart 형식이 올바르지 않은 경우 400 Bad Request 로 응답한다.
- 정상적으로 새 파일을 저장한 경우 201 Created 로 응답한다.

1.3. Concurrency test 를 위한 debug option

동일한 target file에 여러 worker가 동시에 접근할 때 file write operation이 올바르게 synchronized되는지 확인할 목적으로 --slowDown <milli-seconds> option을 구현한다. 이 option은 PUT 또는 POST worker가 synchronized file write 영역에 진입한 뒤 실제 file write를 수행하기 전에 지정된 시간만큼 sleep 하도록 하는 테스트 전용 기능이다. 본 과제에서는 타겟

디렉토리에 대한 file write operation이 상호 배제되도록 synchronized method 또는 synchronized block을 적용하는 것으로 충분하다. 즉, wait/notify를 이용한 fine-grained thread 동기화를 구현하지 않아도 된다. 수행 시 --slowDown option을 명시하지 않은 경우에는 인위적인 지연을 적용하지 않는다.

테스트 시에는 동일한 target filename에 여러 PUT/POST request를 일시에 전송한다. 한 thread가 synchronized write 영역을 사용하는 동안 다른 worker는 해당 영역에 진입하지 못하고 대기해야 한다. Debug mode에서는 synchronized write 영역의 진입과 종료 시 worker thread 이름과 target filename을 console에 출력하도록 한다.

테스트는 아래와 같은 절차로 진행될 수 있다.

1. ./docs/upload/shared.bin 이 존재하지 않는 상태에서 시작한다.
2. 서버를 java MyWebServer --port 8080 --slowDown 500 으로 실행한다.
3. concurrency_test.sh 를 실행하여 동일한 shared.bin 을 target 으로 하는 PUT request 10 개를 동시에 전송한다.
4. 모든 request 가 정상 완료되고, server console 에서 동일한 target file 의 synchronized write 영역에 worker 들이 순차적으로 진입하는 것을 확인한다.

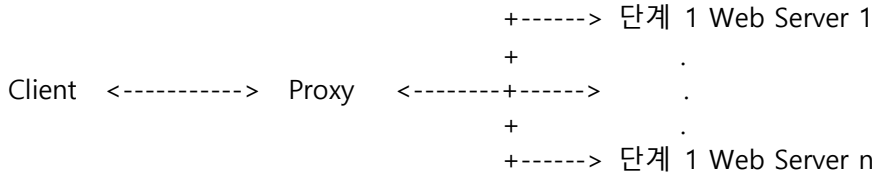
Shell script (concurrency_test.sh)

```
#!/usr/bin/env bash
URL="http://localhost:8080/upload/shared.bin"
FILE="sample.bin"
for i in $(seq 1 10); do
    curl -sS -o /dev/null \
        -w "request=${i} status=%{http_code} time=%{time_total}\n" \
        -X PUT \
        -H "Content-Type: application/octet-stream" \
        --data-binary @"$FILE" \
        "$URL" &
done
wait
```

단계 2: HTTP Reverse Proxy 구현

2.1 Proxy 기본 동작

웹 서버 앞단에서 클라이언트의 요청을 받아 후단의 실제 웹 서버(단계 1에서 구현한 웹 서버를 그대로 이용)로 전달해 주는 proxy 기능을 구현한다. 이를 위해 단계 1에서 개발한 concurrent Web server를 확장하여 reverse proxy 기능을 구현하도록 한다.



1. 프록시 서버는 클라이언트에게는 웹 서버처럼 동작하고, 백엔드 서버에게는 클라이언트처럼 보이도록 작동한다. 프록시 서버는 8080 포트를 사용하고, 후단의 웹 서버는 9001, 9002, 9003 포트를 사용하도록 한다.
2. 선택한 backend에 request를 전달할 때 Host header를 해당 backend의 host/port로 변경하고, X-Forwarded-For: <client IP> header를 추가한다.
3. PUT 또는 POST처럼 request body가 있는 경우 Content-Length에 지정된 byte 수만큼 body를 읽어 backend에 byte stream으로 전달한다.
4. Backend 서버가 반환한 status line, response headers, body를 client에 중계한다.
5. Backend server를 여러 개 실행하고 Proxy가 round robin 방식으로 request를 분산하는 간단한 L7 load balancer 기능을 구현한다.

개발 프로그램을 "java MyWebProxy --port 8080 --server 9001,9002,9003" command로 수행하면 웹 프록시는 localhost:8080에서 동작하게 된다. 실제로 3개의 웹 서버는 각각 localhost:9001, localhost:9002, localhost:9003에서 동작하는데, 이들은 별도의 "java MyWebServer --port portNo" command에 의해 수행된다.

2.2 L7 Load Balancer 기능

(1) Round robin 방식의 load balancing

백엔드 서버 리스트(localhost:9001, localhost:9002, localhost:9003)를 배열로 관리한다. 요청이 들어 올 때마다 9001 -> 9002 -> 9003 -> 9001 ... 순으로 요청을 전달한다. 이 때 synchronized block 또는 thread-safe 카운터(ex, AtomicInteger)를 사용하여 thread 간 race condition이 발생하지 않도록 유의한다. Round robin 동작 확인을 위해 proxy는 request를 forwarding할 때 선택한 backend port, method, target을 console에 출력하고, backend Web server는 수신한 request의 listen port, method, URI, Host, X-Forwarded-For 값을 console에 출력한다. 단계 2 기능 테스트에서는 이 log를 이용하여 request가 9001 -> 9002 -> 9003 순으로 분산되는지와 proxy가 변경한 request header가 backend에 실제로 전달되었는지 확인한다.

(2) Health check and fault failover

Proxy 내부에 별도의 background thread를 두어 약 5초마다 backend server 9001, 9002, 9003에 GET /health request를 전송한다. 이를 위해 GET /health에 대해 200 OK response message를 반환하도록 단계 1의 Web server는 확장되어야 한다. Connection 실패, response timeout 또는 HTTP 5xx response가 발생하는 backend는 unhealthy 상태로 판단하여 round robin 대상에서 일시적으로 제외한다. 이후 다음 health check 싸이클에서 다시 200 OK가 확인되면 해당 인스턴스를 round robin 대상에 다시 포함한다. 사용 가능한 backend 서버가 하나도 없으면 Proxy는 client에게 503 Service Unavailable을 반환한다.

(3) Sticky session 구현

첫 request에서 round robin 방식으로 backend instance를 선택한 뒤 Proxy response에 Set-Cookie: PROXY_ROUTE=<backend port>; Path=/ header를 추가한다. Path=/는 이 cookie가 proxy의 모든 URI path에 대해 전송되도록 적용 범위를 지정한다. 이후 client가 Cookie: PROXY_ROUTE=<backend port>를 보내면 Proxy는 round robin을 수행하지 않고 해당 backend로 request를 전달한다. Cookie가 지정한 backend가 health check에 의해 unhealthy 상태로 제외되어 있으면 다른 backend instance로 자동 failover하지 않고 500 Internal Server Error를 반환한다.

3. 과제 테스트

기능 테스트는 Postman을 기본 도구로 사용하고, 동시성 기능 테스트는 주어진 shell script를 이용한다. GET/PUT/POST request를 전송한 뒤에는 Postman Console에서 실제 전송된 method, URL, request header와 body를 확인한다. Postman이 자동으로 생성한 Content-Length, multipart/form-data의 Content-Type 및 boundary도 이 화면에서 확인할 수 있다. 단, Postman Console에서 확인되는 것은 Postman client가 Web server 또는 Proxy로 전송한 request이며, Proxy가 backend로 다시 구성하여 전달한 request는 Proxy와 backend Web server의 console log에서 확인한다.

단계 #1 기능 테스트

ID	기능 테스트	확인 항목
WS-01	GET /index.html	200, Content-Type, Content-Length, body 내용
WS-02	PUT /upload/sample.bin	201 Created 또는 overwrite 시 200 OK, 저장 파일 확인
WS-03	POST /upload Postman Body → form-data key: file type: File value: sample.png	201 Created, filename="sample.png"로 저장 확인
WS-04	concurrency_test.sh 실행	동일 target file의 write가 synchronized 처리 확인

단계 #2 기능 테스트

ID	기능 테스트	확인 항목
PX-01	PROXY_ROUTE Cookie 제거 후 Proxy 경유 GET 6회	Proxy/Backend console log에서 선택된 backend가 9001 → 9002 → 9003 순으로 두 번 반복되는지 확인
PX-02	Backend 9002 종료 → health check cycle 대기 → Proxy GET 반복 → 9002 재실행 → health check cycle 대기 → Proxy GET 반복	장애 시 9002 제외, 복구 후 round robin 대상에 다시 포함되는지 console log로 확인
PX-03	Cookie 없는 첫 GET 요청 → Set-Cookie 확인 → 동일 cookie로 GET 반복	Postman Console에서 PROXY_ROUTE Cookie 전송을 확인하고, Proxy/Backend console log에서 동일 backend로 전달되는지 확인

4. 과제 제출 방법

과제는 HYON LMS에 구현 소스 코드와 함께 "디자인 및 구현 문서" 파일을 포함하여 제출한다. 이 파일에는 제출 과제를 수행하는 방법이 정확하게 설명되어야 하며, 기술된 방법에 따라 수행하였음에도 불구하고 수행이 되지 않는 경우에는 미제출로 간주한다. 또한 과제를 부분적으로 구현한 경우에는 구현된 기능들을 제출 문서에 명시하여야 한다.

제출 파일명을 Assignment #1은 "Assignment1_학번_이름.zip", Assignment #2는 "Assignment2_학번_이름.[jar|zip]"으로 하여야 한다. java 파일을 반드시 포함해야 하며, class 파일 혹은 executable 파일이 포함되지 않도록 유의한다.

소스 코드 제출 시한은 Assignment #1은 11월 16일 월요일 자정까지, Assignment #2는 11월 30일 월요일 자정까지로 설정한다. 지연된 프로그램 제출의 감점은 제출 시한으로부터 24 시간 이내는 50% 감점, 48시간 이내는 75% 감점이다. 제출 시한으로부터 48시간 이후 제출은 점수가 주어지지 않는다.

디자인 및 구현 문서에서는 프로그램의 설계 및 구조의 기술 외에도 제출된 프로그램을 컴파일 및 실행하는 방법을 단계별로 정확하게 기술하여야 한다. 또한, 소스 코드를 단순히 Copy&Paste하여 의미없는 문서를 제출하는 경우 감점의 요소가 됨을 유의하여야 한다. 프로그램 테스트 시 기술된 절차를 그대로 따라 했음에도 불구하고 프로그램이 수행되지 않는 경우에는 제출 프로그램이 동작하지 않는 것으로 간주한다

4.1 디자인 및 구현 문서에 포함할 내용

단계 #1 디자인 및 구현 문서

1. 전체 source directory 구조와 주요 class 설명
2. HTTP request parsing과 response 메시지 작성 방법
3. Thread-safe 프로그램 설계, 쓰레드 동기화, synchronized access 기법에 대한 설명
4. GET/PUT/POST URI와 file upload/download 처리 방법
5. 공유 data structure와 공유 디렉토리에 file upload synchronization 보장 방법
6. 단계 #1 기능 테스트 결과 (캡쳐 포함)

단계 #2 디자인 및 구현 문서

1. 전체 source directory 구조와 주요 class 설명
2. Proxy 구조와 request/response forwarding 설명
3. Round robin, health check, sticky session 상태 관리 방법
4. 단계 #2 기능 테스트 결과 (캡쳐 포함)

4.2 권장 제출 directory 구조

```
Assignment*_학번_이름/
 ├─── src/
 |       └─── *.java
 ├─── docs/
 |       ├─── public/
 |       └─── upload/
 +─── 디자인및구현문서.pdf
```

4.3 테스트 도구 및 방법

프로그램 개발을 위한 Java IDE는 IntelliJ IDEA로 제한된다. 프로그램 개발 과정에서 클라이언트 프로그램으로 일반 웹 브라우저, curl, Postman(https://www.postman.com) 등을 사용할 수 있다. 과제의 기능 테스트는 **Postman을 기본 도구로 사용**하며, 동일 target file에 대한 concurrency test는 과제와 함께 제공되는 **concurrency_test.sh를 사용**한다. 디자인 및 구현 문서에는 각 단계에서 요구하는 기능 테스트의 주요 결과, Postman Console에서 확인한 request 정보 및 실행 화면을 포함하여야 한다.